

Post-Quantum Signcryption for Secure IoT Data Stream Processing in Cloud-based Hadoop Systems

Krit Meesiri, Panod Manodee, Apichat Laohawan, Sujitta Sonna

Sirindhorn International Institute of Technology, Thammasat University, Thailand

6622780037@g.siit.tu.ac.th, 6622782033@g.siit.tu.ac.th, 6622770491@g.siit.tu.ac.th, 6622770640@g.siit.tu.ac.th

Abstract—The integration of high-frequency Internet of Things (IoT) data streams with cloud-based big data platforms, such as Hadoop, introduces significant challenges in achieving scalable and end-to-end secure data processing. Existing solutions either impose prohibitive cryptographic overhead on resource-constrained IoT devices or fail to provide comprehensive protection against emerging quantum threats. In particular, applying post-quantum cryptography at the device level is impractical, while per-record cryptographic processing in high-throughput stream environments leads to severe scalability limitations.

This paper proposes a secure and scalable IoT stream processing framework that integrates a hybrid post-quantum signcryption mechanism into a Hadoop-based architecture. The framework adopts a layered design in which IoT devices perform lightweight authenticated encryption combined with PUF-based authentication, while all post-quantum operations are offloaded to a clustered edge layer composed of Trusted Execution Environment (TEE)-protected enclaves. Apache Kafka and Spark Structured Streaming are employed to perform cryptography-aware preprocessing, including epoch-based batching, deduplication, and ciphertext indexing, without exposing plaintext data. At the core of the proposed system is an epoch-based hybrid post-quantum signcryption construction that unifies CRYSTALS-Kyber key encapsulation, lattice-based CP-ABE, and CRYSTALS-Dilithium signatures into a single protection pipeline. This design simultaneously achieves IND-CCA2 confidentiality, fine-grained access control, and EUF-CMA authenticity under standard lattice assumptions. By aggregating records into epochs, the framework reduces expensive lattice-based operations from $\mathcal{O}(n_e)$ to $\mathcal{O}(1)$ per epoch, significantly improving throughput in high-rate IoT environments. Furthermore, the clustered TEE architecture enables parallel and isolated execution of cryptographic operations, enhancing both scalability and security. The proposed framework provides an end-to-end solution for secure, quantum-resistant IoT data stream processing, enabling efficient storage, retrieval, and verification over encrypted data in distributed cloud environments.

I. INTRODUCTION

The Internet of Things (IoT) has become a fundamental component of modern digital infrastructure, enabling continuous data generation across domains such as industrial automation, smart cities, and healthcare. These large-scale data streams are typically integrated into distributed big data platforms, where frameworks such as Hadoop, Kafka, and Spark provide scalable storage and real-time processing capabilities [2], [11], [26]. However, the convergence of high-frequency IoT data streams with cloud-based big data systems introduces significant challenges in achieving *end-to-end security, scalability, and efficiency*.

A core challenge lies in the mismatch between *strong security requirements* and *resource-constrained IoT devices*. IoT data often contains sensitive information that demands confidentiality, integrity, authentication, and fine-grained access control. At the same time, IoT devices are inherently limited in computation, memory, and energy, making them unsuitable for frequent public-key cryptographic operations. This challenge becomes critical in streaming environments, where data is generated continuously and per-record cryptographic processing introduces substantial latency and computational overhead.

The problem is further intensified by the emergence of quantum computing. Classical public-key cryptosystems such as RSA and ECC are vulnerable to quantum attacks (e.g., Shor's algorithm), exposing current systems to *harvest-now-decrypt-later* threats. To address this, NIST has standardized post-quantum cryptographic primitives such as CRYSTALS-Kyber and CRYSTALS-Dilithium. While these lattice-based schemes provide strong security guarantees, their direct deployment in IoT environments remains impractical due to large key sizes and high computational complexity [9], [10].

Recent studies have explored lattice-based access control mechanisms [1], [32] and post-quantum signcryption schemes [15], [16], [21], demonstrating the feasibility of quantum-resistant designs. However, these works primarily focus on standalone cryptographic primitives or isolated protocols. They do not address how such mechanisms can be efficiently integrated into high-throughput IoT data pipelines. Furthermore, most existing approaches rely on *per-message cryptographic operations*, which scale poorly in streaming scenarios.

In parallel, secure data processing frameworks have been developed for IoT and big data systems. Streaming platforms such as Kafka and Spark enable efficient ingestion and processing [2], while Hadoop-based systems support large-scale encrypted storage [26]. Lightweight encryption techniques such as ChaCha20-Poly1305 [14] and Trusted Execution Environments (TEE) [8] have been employed to enhance security. Nevertheless, these systems predominantly rely on classical cryptography and do not provide post-quantum protection. Moreover, security mechanisms such as authentication, encryption, and access control are typically implemented in a *disjoint manner*, leading to inefficiencies and increased system complexity.

From the above discussion, existing solutions suffer from

several critical limitations:

- **Lack of end-to-end post-quantum integration:** Most works focus on individual primitives rather than unified system-level designs.
- **Inefficient per-record cryptographic processing:** Existing schemes incur $\mathcal{O}(n)$ overhead for n data records, limiting scalability in high-rate IoT streams.
- **Disjoint security mechanisms:** Authentication, encryption, and access control are handled separately, resulting in redundant computations.
- **Limited support for streaming environments:** Prior works do not fully integrate cryptographic protection with real-time data pipelines.

To address these challenges, this paper proposes a *secure, scalable, and quantum-resistant IoT stream processing framework* that integrates a hybrid post-quantum signcryption mechanism into a Hadoop-based architecture. The proposed system adopts a layered IoT-edge-cloud design. At the IoT layer, devices perform lightweight authenticated encryption combined with PUF-based authentication, avoiding expensive public-key operations. At the streaming layer, Kafka and Spark Structured Streaming organize incoming ciphertexts into epoch-aligned batches and perform cryptography-aware preprocessing. At the edge layer, a cluster of TEE-protected enclaves executes post-quantum cryptographic operations in parallel, ensuring both strong security and high throughput.

The core of the proposed framework is an *epoch-based hybrid post-quantum signcryption construction* that tightly integrates CRYSTALS-Kyber key encapsulation, lattice-based CP-ABE, and CRYSTALS-Dilithium signatures into a unified pipeline. Unlike conventional designs that treat encryption and signing as separate steps, the proposed construction achieves confidentiality, fine-grained access control, and authenticity in a single cohesive operation. Furthermore, by shifting from per-record processing to epoch-based batching, the framework reduces expensive lattice-based operations from $\mathcal{O}(n_e)$ to $\mathcal{O}(1)$ per epoch, enabling practical deployment in high-throughput IoT environments.

The main contributions of this paper are summarized as follows:

- **Context-bound hybrid batch protection for post-quantum IoT streams.** We design a context-bound hybrid protection workflow that integrates Kyber-based key encapsulation, lattice CP-ABE, and Dilithium signatures within a batch-oriented edge-side processing model. The design binds each epoch key to both batch content and policy context, enabling confidentiality, integrity, and fine-grained access control with reduced cryptographic overhead.
- **Epoch-based batch cryptographic model.** We introduce an *epoch-based signcryption paradigm* that shifts cryptographic processing from per-record to per-batch execution, reducing computational complexity from $\mathcal{O}(n_e)$ to $\mathcal{O}(1)$ per epoch. This significantly improves scalability

and makes post-quantum protection practical for high-throughput IoT data streams.

- **TEE-assisted edge offloading architecture.** We design a clustered TEE-based edge architecture that securely offloads all post-quantum cryptographic operations from resource-constrained IoT devices while enabling parallel and isolated execution. This approach effectively balances strong security guarantees with system performance.
- **Cryptography-aware streaming pipeline.** We develop a secure streaming framework based on Kafka and Spark Structured Streaming that performs batching, deduplication, metadata tokenization, and ciphertext indexing directly over encrypted data. This enables efficient processing and retrieval without exposing plaintext information.

II. RELATED WORK

This section reviews existing studies in three main categories: (i) post-quantum cryptography and access control for IoT, (ii) signcryption schemes including post-quantum constructions, and (iii) secure data processing frameworks for IoT and big data systems.

A. Post-Quantum Cryptography and Access Control in IoT

The rapid advancement of quantum computing has driven the development of post-quantum cryptographic (PQC) mechanisms for IoT systems. Lattice-based constructions, particularly those based on the Learning With Errors (LWE) assumption, have emerged as strong candidates due to their resistance to quantum attacks. Zhang *et al.* [1] proposed a lattice-based puncturable CP-ABE scheme with forward security for cloud-assisted IoT, demonstrating the feasibility of integrating fine-grained access control with post-quantum security. Similarly, recent studies have explored efficient implementations of CRYSTALS-Dilithium [10] and verifiable post-quantum signatures [9].

Beyond cryptographic primitives, access control models such as ABAC and break-the-glass mechanisms have been widely studied for IoT and healthcare systems [4], [5], [6], [7]. These approaches provide flexible and context-aware authorization but are typically designed for classical cryptographic settings. Recent work has extended these models into the post-quantum domain, incorporating lattice-based access control and efficient revocation mechanisms [32].

Despite these advancements, existing solutions primarily focus on standalone cryptographic primitives or static data protection. They do not address the integration of PQC into high-throughput IoT data streams, nor do they consider the scalability challenges introduced by continuous data generation and real-time processing.

B. Signcryption and Post-Quantum Signcryption Schemes

Signcryption provides an efficient mechanism to achieve confidentiality and authenticity simultaneously, outperforming traditional encrypt-then-sign approaches. Early works applied signcryption to cloud and big data systems for secure storage and communication [19], [25], [30].

Recent efforts have extended signcryption into the post-quantum setting. Sinha *et al.* [15] introduced a lattice-based identity-based ring signcryption scheme for IoT devices, while Xu *et al.* [16] proposed a certificateless construction with linkability. Other variants include blind signcryption [17], broadcast signcryption [18], lattice-based anonymous schemes [21], and threshold or blockchain-assisted designs [22], [29]. Additionally, outsourced and hybrid signcryption approaches have been proposed to reduce computational overhead on resource-constrained devices [28], [33].

However, several limitations remain. First, many schemes rely on loosely coupled encryption and signature components rather than a tightly integrated design. Second, most constructions operate at the per-message level, leading to significant computational overhead in high-frequency IoT environments. Third, these schemes are typically evaluated in isolation and lack integration with real-world data processing frameworks such as streaming or Hadoop-based systems.

C. Secure IoT Data Processing and Big Data Frameworks

The integration of IoT data streams with big data platforms such as Hadoop has been widely explored to support scalable storage and analytics. Kafka and Spark Structured Streaming provide efficient pipelines for real-time data ingestion and processing [2], while Hadoop-based systems have been enhanced with security mechanisms including authentication and key exchange protocols [11] and large-scale encryption techniques [26].

In IoT environments, lightweight encryption schemes such as ChaCha20-Poly1305 have been adopted to accommodate resource constraints [14]. Trusted Execution Environments (TEE) have also been utilized to protect sensitive operations in untrusted environments [8], enabling secure computation at the edge layer.

Nevertheless, most existing frameworks rely on classical cryptographic techniques and do not address post-quantum threats. Furthermore, cryptographic protection is typically applied at the individual record level, resulting in scalability bottlenecks in high-throughput streaming scenarios. Existing systems also treat authentication, encryption, and access control as separate components, leading to inefficiencies and increased system complexity.

III. OUR PROPOSED SYSTEM

This section presents the system model, threat model, and cryptographic construction of the proposed secure IoT data streaming framework. The objective is to protect large-scale IoT data generated by resource-constrained devices against both classical and quantum adversaries.

The proposed architecture integrates PUF-based device authentication with edge-assisted post-quantum signcryption to achieve secure, scalable, and efficient data protection. It supports secure data transmission, integrity verification, and fine-grained access control prior to storage in the Hadoop Distributed File System (HDFS).

A. System Model

The proposed system consists of six main entities: *IoT devices*, *Kafka ingestion layer*, *Spark streaming engine*, *edge protection cluster*, *attribute authority*, and *HDFS storage*. These components collaboratively enable secure, scalable, and post-quantum-resilient IoT data stream processing.

1) *IoT Devices*: IoT devices are responsible for sensing and generating data continuously. Due to their limited computational capabilities, they perform only lightweight operations, including symmetric encryption and simple authentication token generation. Each device protects its data before transmission to ensure confidentiality and integrity while avoiding expensive public-key computations.

2) *PUF-Based Device Authentication*: Each IoT device is equipped with a Physical Unclonable Function (PUF) used to generate hardware-bound authentication tokens. These tokens provide lightweight and tamper-resistant device authentication without relying on digital signatures. This mechanism ensures that only legitimate devices can inject data into the system.

3) *Kafka Ingestion Layer*: Kafka acts as a high-throughput and fault-tolerant message broker that collects encrypted data streams from IoT devices. It decouples data producers and consumers, ensuring reliable ingestion and buffering of large-scale streaming data before further processing.

4) *Spark Streaming Engine*: Spark Structured Streaming processes incoming data streams in near real time. It organizes incoming records into epoch-based batches, performs preprocessing operations such as deduplication and metadata extraction, and associates each batch with a corresponding access control policy. This batching mechanism is critical for improving scalability and reducing cryptographic overhead in later stages.

5) *Edge Protection Cluster (TEE-Based)*: The edge layer consists of multiple edge nodes, each hosting one or more Trusted Execution Environment (TEE) enclaves. This layer performs all computationally intensive and security-critical operations. Specifically, it verifies device authenticity, manages key establishment, and executes post-quantum cryptographic operations, including hybrid signcryption and policy enforcement. The use of TEE ensures that sensitive operations and intermediate secrets remain protected even in untrusted environments. The cluster-based design further enables load balancing and parallel processing across multiple enclaves.

6) *Attribute Authority*: The Attribute Authority (AA) is responsible for managing access control policies. It initializes the attribute-based encryption system and issues private keys to users based on their attributes. This enables fine-grained, policy-driven access control over encrypted data stored in the system.

7) *HDFS Storage*: HDFS serves as the persistent storage layer for encrypted IoT data. It stores protected data batches along with their associated metadata and indexes to support efficient retrieval. HDFS also functions as the final receiver of encrypted keys and data, ensuring that only authorized entities can decrypt and access the stored information.

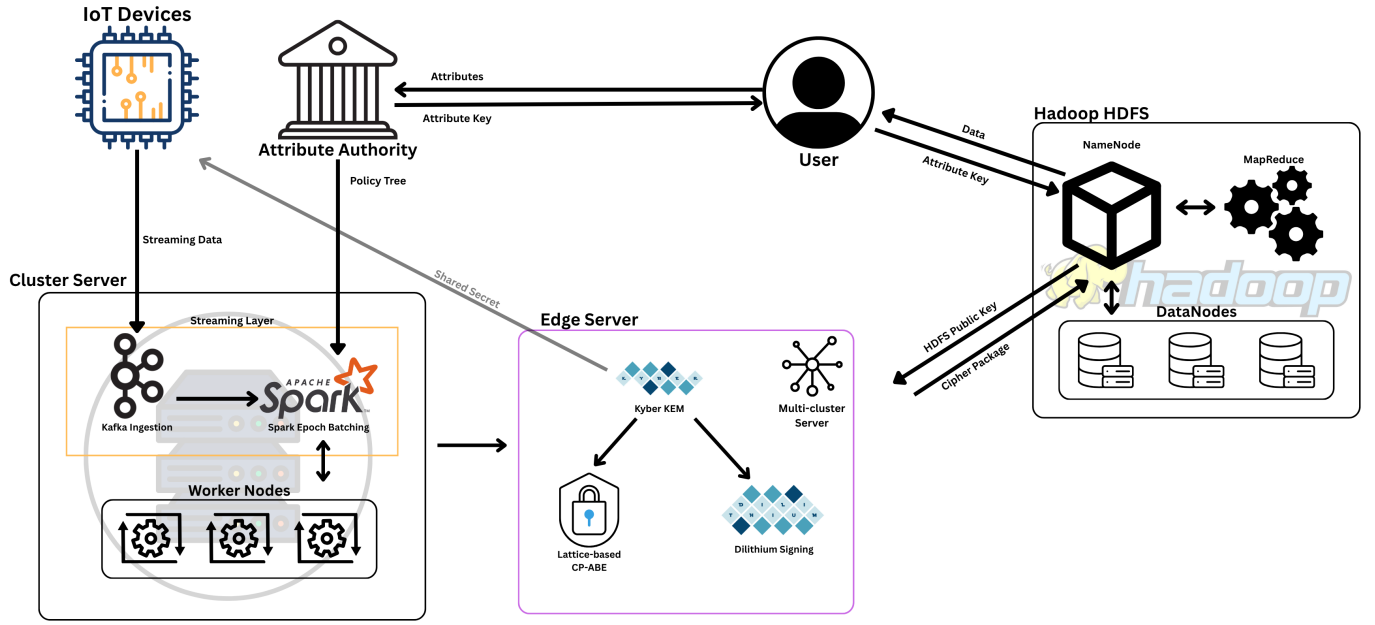


Fig. 1. System Model

B. Threat Model

We consider a computationally bounded adversary $\mathcal{A}$ operating under a Dolev–Yao network model, with additional capabilities reflecting modern cloud and post-quantum threat settings. The adversary is assumed to have full control over the communication channels but cannot break standard cryptographic primitives except with negligible probability.

a) *Adversarial Capabilities*: The adversary $\mathcal{A}$ may perform the following actions:

- **Eavesdropping**: $\mathcal{A}$ can passively observe all communications between IoT devices, Kafka brokers, Spark streaming nodes, edge servers, and HDFS storage.
- **Message Injection and Modification**: $\mathcal{A}$ can inject fabricated messages, alter transmitted ciphertexts, or reorder messages within the network.
- **Replay Attacks**: $\mathcal{A}$ may replay previously transmitted records in an attempt to disrupt system correctness or bypass authentication.
- **Unauthorized Data Access**: $\mathcal{A}$ may attempt to decrypt stored ciphertext or recover plaintext data without possessing valid attribute-based credentials.
- **Chosen-Ciphertext Interaction**: $\mathcal{A}$ may adaptively interact with the system by submitting crafted ciphertexts or queries to gain information about encrypted data.
- **Quantum Adversary**: $\mathcal{A}$ may possess quantum computational capabilities and can exploit algorithms such as Shor’s algorithm to break classical public-key cryptosystems.

b) *Trust Assumptions*: The following components are assumed to be trusted:

- **TEE Enclaves**: Trusted Execution Environments are assumed to provide secure and isolated execution. The

adversary cannot access enclave memory or internal cryptographic states.

- **Attribute Authority (AA)**: The AA is trusted to correctly generate and distribute attribute-based secret keys and is not compromised.
- **HDFS Receiver Keys**: The secret key SK_S used for Kyber decapsulation is securely stored and not exposed to the adversary.

The following components are considered *honest-but-curious* or potentially exposed:

- **Network Infrastructure**: Communication channels and intermediate nodes (e.g., Kafka and Spark) are untrusted and may be monitored or manipulated.
- **Storage Layer**: HDFS is assumed to be honest-but-curious and may observe stored ciphertexts but cannot decrypt them.

TABLE I
MAIN NOTATIONS USED IN THE CRYPTOGRAPHIC PROCESS

Symbol	Description
D_i	i -th IoT device
ID_i	Identity of device D_i
m_i	Plaintext sensing data generated by D_i
ts_i	Timestamp of record i
ctr_i	Monotonic counter of device D_i
$topic_i$	Sensor class or application topic
K_i	Symmetric encryption key for record i
$nonce_i$	AEAD nonce for record encryption
AAD_i	Associated authenticated data
C_i	Ciphertext generated from m_i
ce	Epoch challenge used for PUF authentication
$r_{i,e}$	PUF response of device D_i for epoch e
$Auth_i$	PUF-based authentication token
M_i	Protected message generated by the IoT device
Δ	Epoch duration
e	Epoch identifier

C. Cryptographic Process

This section presents the end-to-end cryptographic workflow of the proposed framework. The system is designed to support secure and scalable IoT data stream processing while minimizing cryptographic overhead on resource-constrained devices. To achieve this objective, the workflow is decomposed into multiple sequential phases spanning the IoT layer, the stream processing layer, the edge protection layer, and the secure storage layer.

Before describing the workflow, Table III summarizes the main notations used throughout the cryptographic process.

The workflow consists of five phases. Phase 1 performs system initialization and key establishment. Phase 2 provides lightweight protection at IoT devices. Phase 3 performs stream ingestion and epoch-based batching using Kafka and Spark. Phase 4 applies hybrid post-quantum signcryption at the edge server, and Phase 5 supports secure storage and retrieval in Hadoop/HDFS.

1) *Phase 1: Initialization*: Phase 1 is executed by the HDFS receiver and the edge protection server. Its objective is to initialize cryptographic keys and establish per-device shared secrets that enable secure data transmission from IoT devices to the cloud through the edge layer.

a) *Step 1: Lattice-Based Key Pair Generation*: The HDFS receiver generates a lattice-based public/secret key pair using the Kyber key encapsulation mechanism:

$$(PK_S, SK_S) \leftarrow \text{Kyber.KeyGen}(1^\lambda), \quad (1)$$

where λ is the security parameter. The public key PK_S is published, while the secret key SK_S is securely retained by HDFS for subsequent decapsulation during data recovery.

b) *Step 2: Device-Bound Key Encapsulation at the Edge*: For each IoT device D_i , the edge server derives a device-specific context

$$u_i = (ID_i, t_i), \quad (2)$$

where ID_i denotes the device identity and t_i represents the timestamp at which the shared secret is generated.

Using the HDFS public key, the edge server performs key encapsulation:

$$(CTK_i, SS_i) \leftarrow \text{Kyber.Encaps}(PK_S, u_i), \quad (3)$$

where SS_i is the resulting shared secret associated with device D_i , and CTK_i is the corresponding encapsulation ciphertext.

The shared secret SS_i serves as the root key for subsequent symmetric key derivation at the IoT device layer. The edge server securely provisions SS_i to device D_i through a protected channel established during device registration.

To ensure key freshness, each shared secret is associated with a validity period derived from t_i . Upon expiration, the edge server re-executes the encapsulation process to generate a new (CTK_i, SS_i) pair, thereby replacing the previous key material.

The encapsulation ciphertext CTK_i is retained for use in later phases, where it is incorporated into the epoch-level protection process.

2) *Phase 2: Lightweight IoT Data Encryption with PUF-Based Authentication*: Phase 2 is executed entirely at the IoT device layer. Its objective is to collect sensing data, protect the payload using lightweight authenticated encryption, and generate a PUF-bound authentication token for efficient device authentication in subsequent phases. To accommodate resource-constrained devices, this phase avoids public-key cryptographic operations and relies only on symmetric cryptography, hashing, and PUF evaluation.

a) *Step 1: Data Acquisition*: Let D_i denote the i -th IoT device. At time ts_i , device D_i collects a sensing measurement m_i . Each record is associated with a monotonically increasing counter ctr_i , which preserves ordering and supports replay detection.

The raw sensing record is defined as

$$R_i = (ID_i, ts_i, ctr_i, topic_i, m_i), \quad (4)$$

where ID_i is the device identity and $topic_i$ denotes the application or sensor category.

b) *Step 2: Lightweight Key Derivation and Nonce Generation*: Using the device-specific shared secret SS_i established in Phase 1, the device derives a record-level symmetric key:

$$K_i \leftarrow \text{KDF}(ID_i \parallel ts_i \parallel ctr_i, SS_i), \quad (5)$$

$$nonce_i \xleftarrow{\$} \{0, 1\}^{96}. \quad (6)$$

This construction ensures that each record is protected with a unique key derived from both device identity and temporal context, enabling fine-grained key separation across records.

c) *Step 3: Construction of Associated Authenticated Data*: The device constructs associated authenticated data (AAD) from non-sensitive metadata:

$$AAD_i = ID_i \parallel ts_i \parallel ctr_i \parallel topic_i. \quad (7)$$

Although not encrypted, AAD_i is cryptographically bound to the ciphertext, ensuring integrity of critical metadata fields.

d) *Step 4: Authenticated Encryption of Sensing Data:* The sensing data m_i is encrypted using ChaCha20-Poly1305:

$$C_i = \text{Enc}_{\text{ChaCha20-Poly1305}}(K_i, \text{nonce}_i, m_i, \text{AAD}_i). \quad (8)$$

The resulting ciphertext C_i provides confidentiality and integrity for both payload and associated metadata.

The protected record is then represented as

$$\tilde{R}_i = (ID_i, ts_i, ctr_i, topic_i, nonce_i, \text{AAD}_i, C_i). \quad (9)$$

e) *Step 5: Epoch-Aware PUF-Based Authentication:* To authenticate the origin of each record without incurring public-key overhead, the device employs a Physical Unclonable Function (PUF). Given the epoch challenge c_e , device D_i computes

$$r_{i,e} = \text{PUF}_i(c_e). \quad (10)$$

The authentication token is constructed as

$$\text{Auth}_i = H(ID_i \parallel ts_i \parallel ctr_i \parallel c_e \parallel r_{i,e} \parallel H(C_i)). \quad (11)$$

This design binds the authentication token to the device identity, temporal context, and ciphertext content, ensuring that any modification or replay of records invalidates the token.

f) *Step 6: Formation of the Protected Message:* Finally, the device constructs the outgoing message:

$$M_i = (ID_i, ts_i, ctr_i, topic_i, nonce_i, \text{AAD}_i, C_i, c_e, \text{Auth}_i). \quad (12)$$

The message M_i is transmitted to the stream ingestion layer for further processing.

g) *Design Insight and Novelty:* This phase introduces a lightweight yet expressive protection mechanism tailored for IoT environments. First, the use of context-aware key derivation enables per-record key separation without requiring additional key exchanges. Second, the integration of epoch-aware PUF authentication allows device-origin verification without digital signatures, significantly reducing computational overhead. Third, by binding ciphertext hashes and temporal metadata into the authentication token, the design enables efficient batch verification in later phases while preserving strong data integrity guarantees.

Phase 2 therefore establishes a secure and scalable foundation for subsequent stream processing and edge-side cryptographic protection.

3) *Phase 3 — Stream Ingestion and Epoch-Aware Cryptographic Preprocessing:* Phase 3 is executed within the streaming layer, consisting of Kafka and Spark Structured Streaming. Its objective is to ingest protected messages from Phase 2, organize them into epoch-aligned batches, and perform cryptography-aware preprocessing to reduce the cost of downstream post-quantum operations. Unlike conventional pipelines, this phase is not merely a transport layer, but a computation-aware stage that transforms high-frequency ciphertext streams into compact, verifiable batch representations.

a) *Input from Phase 2:* Each IoT device produces protected messages of the form

$$M_i = (ID_i, ts_i, ctr_i, topic_i, nonce_i, \text{AAD}_i, C_i, c_e, \text{Auth}_i), \quad (13)$$

where C_i is the AEAD ciphertext and Auth_i is the PUF-based authentication token. These messages are published to Kafka topics according to deployment-specific routing policies.

b) *Step 1: Kafka-Based Stream Ingestion:* Kafka provides a high-throughput buffering and decoupling layer. The incoming stream is denoted by

$$\mathcal{S} = \{M_1, M_2, \dots\}. \quad (14)$$

Messages are partitioned using a routing function:

$$p_i = \text{PartFn}(ID_i, topic_i, region_i), \quad (15)$$

which enables parallel processing while preserving locality for subsequent batch formation. Kafka does not perform any decryption or cryptographic transformation, ensuring that confidentiality and integrity from Phase 2 remain intact.

c) *Step 2: Metadata Parsing Without Decryption:* Spark consumes the stream and extracts only non-sensitive metadata:

$$\text{Meta}(M_i) = (ID_i, ts_i, ctr_i, topic_i, c_e). \quad (16)$$

The ciphertext C_i remains opaque. This design enables scheduling, batching, and indexing without exposing plaintext data.

d) *Step 3: Epoch-Based Micro-Batch Formation:* To enable scalable downstream protection, Spark assigns each message to an epoch:

$$e_i = \left\lfloor \frac{ts_i}{\Delta} \right\rfloor, \quad (17)$$

and groups records into batches:

$$\mathcal{B}_e = \{M_i \in \mathcal{S} \mid e_i = e\}. \quad (18)$$

This epoch-based aggregation is a key optimization. Instead of applying post-quantum operations per record, the system performs them once per batch. If $n_e = |\mathcal{B}_e|$, the cost of heavy cryptographic operations is reduced from

$$O(n_e) \text{ to } O(1) \quad (19)$$

per epoch, enabling practical scalability for high-rate IoT streams.

e) *Step 4: Deduplication and Stream Normalization:* To eliminate redundant processing, Spark computes a deterministic identifier:

$$\text{RID}_i = H(ID_i \parallel ts_i \parallel ctr_i \parallel topic_i), \quad (20)$$

and removes duplicates within the epoch:

$$\mathcal{D}_e = \{M_i \in \mathcal{B}_e \mid \text{RID}_i \notin \mathcal{S} \setminus \mathcal{D}_e\}. \quad (21)$$

This prevents repeated cryptographic protection and reduces storage overhead.

f) *Step 5: Policy Context Association:* For each batch, Spark derives a context:

$$ctx_e = \text{CtxFn}(\mathcal{D}_e), \quad (22)$$

and maps it to a policy reference:

$$PolRef_e = \text{PolicyMap}(ctx_e). \quad (23)$$

This separates stream classification from access control enforcement, allowing the edge layer to apply CP-ABE policies efficiently.

g) *Step 6: Metadata Tokenization for Encrypted Retrieval:* To support searchable encrypted storage, Spark converts metadata into hashed tokens:

$$kw_{i,1} = H(ID_i), \quad (24)$$

$$kw_{i,2} = H(topic_i), \quad (25)$$

$$kw_{i,3} = H(e_i), \quad (26)$$

$$kw_{i,4} = H(class_i \parallel region_i). \quad (27)$$

These tokens enable ciphertext-aware indexing without revealing plaintext information.

h) *Step 7: Epoch-Aligned Ciphertext Index Construction:* For each accepted record, Spark constructs:

$$ObjID_i = H(ID_i \parallel ts_i \parallel ctr_i \parallel H(C_i)), \quad (28)$$

$$Idx_i = (\{kw_{i,1}, kw_{i,2}, kw_{i,3}, kw_{i,4}\} \rightarrow ObjID_i). \quad (29)$$

The batch index is

$$\mathcal{I}_e = \{Idx_i \mid M_i \in \mathcal{D}_e\}. \quad (30)$$

This index is aligned with epoch batching, enabling efficient retrieval without scanning the full dataset.

i) *Step 8: Pre-Computation of Cryptographic Digests:* To reduce edge-side overhead, Spark computes preliminary digests:

$$h_i^{(0)} = H(ID_i \parallel ts_i \parallel ctr_i \parallel e_i \parallel C_i), \quad (31)$$

$$\mathcal{H}_e^{(0)} = \{h_i^{(0)} \mid M_i \in \mathcal{D}_e\}. \quad (32)$$

These digests enable efficient batch integrity construction in the next phase.

For each epoch e , Spark outputs:

$$\Pi_e = (\mathcal{D}_e, \mathcal{I}_e, \mathcal{H}_e^{(0)}, PolRef_e, e). \quad (33)$$

This package contains the deduplicated ciphertext batch, index structure, pre-computed digests, and policy reference, and is forwarded to the edge server for post-quantum protection.

This phase introduces a cryptography-aware stream processing paradigm. First, epoch-based batching transforms per-record cryptographic cost into constant-time per epoch. Second, preprocessing operations such as deduplication, tokenization, and digest computation are performed without decrypting ciphertexts, preserving end-to-end confidentiality. Third, the alignment between batching, indexing, and policy association enables efficient integration with downstream post-quantum signcryption, significantly improving scalability in high-throughput IoT environments.

4) *Phase 4 — Clustered TEE-Assisted Post-Quantum Signcryption:* Phase 4 is executed at the edge protection layer, which is organized as a cluster of edge nodes, each hosting one or more Trusted Execution Environment (TEE) enclaves. The objective of this phase is to perform secure device authentication, derive epoch-level protection keys, and apply hybrid post-quantum signcryption over batched ciphertext records in a scalable and parallel manner.

Unlike a single-enclave design, the proposed architecture distributes incoming epoch batches across multiple attested enclaves, enabling parallel processing while preserving strong security guarantees. Each enclave operates as an isolated trust domain, ensuring that cryptographic states, keys, and verification processes remain confined.

a) *Input from Phase 3:* For each epoch e , the streaming layer outputs:

$$\Pi_e = (\mathcal{D}_e, \mathcal{I}_e, \mathcal{H}_e^{(0)}, PolRef_e, e). \quad (34)$$

A routing function assigns Π_e (or its partition) to a target enclave:

$$T_{j,k} = \text{RouteFn}(e, ID_i, load_j), \quad (35)$$

where $T_{j,k}$ denotes the k -th enclave on edge node E_j . This function preserves device locality while balancing computational load across the cluster.

b) *Step 1: Enclave Isolation and Secure Loading:* The assigned batch $\Pi_e^{(j,k)}$ is loaded into enclave $T_{j,k}$. All subsequent operations, including authentication verification, key derivation, and signcryption, are executed entirely within the enclave to prevent leakage of sensitive data.

c) *Step 2: PUF-Based Device Authentication:* For each record $M_i \in \mathcal{D}_e^{(j,k)}$, the enclave reconstructs and verifies the authentication token:

$$\hat{Auth}_i = H(ID_i \parallel ts_i \parallel ctr_i \parallel c_e \parallel r_{i,e} \parallel H(C_i)). \quad (36)$$

Only valid records are retained:

$$\mathcal{V}_e^{(j,k)} = \{M_i \mid \hat{Auth}_i = Auth_i\}. \quad (37)$$

This step ensures device authenticity and eliminates replayed or tampered records before further processing.

d) *Step 3: Epoch-Level Key Generation and Binding:* The enclave generates a single symmetric key for the batch:

$$K_e \xleftarrow{\$} \{0, 1\}^{256}. \quad (38)$$

To bind the key to the batch content and policy context, a domain-separated binding value is computed:

$$ctx_e = H("CTX" \parallel e \parallel PolRef_e \parallel H(\mathcal{H}_e^{(0)})). \quad (39)$$

This design ensures that any modification to batch content or policy invalidates subsequent decryption.

e) *Step 4: Batch-Level AEAD Protection*: The verified ciphertexts are aggregated:

$$C_e^{agg} = \text{Concat}(C_i \mid M_i \in \mathcal{V}_e^{(j,k)}), \quad (40)$$

and protected using AEAD:

$$(CT_e, Tag_e) = \text{AES-GCM.Enc}(K_e, IV_e, C_e^{agg}, ctx_e), \quad (41)$$

where IV_e is a unique nonce per epoch.

This step provides an additional confidentiality and integrity layer over the entire batch.

f) *Step 5: Post-Quantum Key Protection via KEM*: The epoch key is protected using Kyber:

$$(CTK_e, SS_e) \leftarrow \text{Kyber.Encaps}(PK_S), \quad (42)$$

$$K'_e = \text{KDF}(SS_e), \quad (43)$$

$$\text{Enc}K_e = K_e \oplus K'_e. \quad (44)$$

Only the HDFS receiver can recover K_e using SK_S , ensuring post-quantum confidentiality of the batch key.

g) *Step 6: Policy-Based Access Control (CP-ABE)*: The enclave maps $PolRef_e$ to an access structure $\mathbb{A}_e$ and enforces fine-grained access control:

$$CT_e^{ABE} = \text{CP-ABE.Enc}(\mathbb{A}_e, \text{Enc}K_e). \quad (45)$$

This enables attribute-based access over encrypted data without exposing the key.

h) *Step 7: Batch Commitment and Integrity Binding*: A final batch digest is constructed:

$$h_e = H(e \parallel CT_e \parallel Tag_e \parallel CT_e^{ABE} \parallel CTK_e). \quad (46)$$

Optionally, a Merkle root over $\mathcal{H}_e^{(0)}$ can be included to support efficient inclusion proofs during verification.

i) *Step 8: Post-Quantum Signature Generation*: The enclave signs the batch digest using Dilithium:

$$\sigma_e \leftarrow \text{Dilithium.Sign}(SK_{sig}^{(j,k)}, h_e). \quad (47)$$

Each enclave uses its own signing key, limiting the impact of potential compromise and enabling parallel signature generation.

j) *Output of Phase 4*: Each enclave outputs a signcrypted batch:

$$\Omega_e^{(j,k)} = (CT_e, Tag_e, CT_e^{ABE}, CTK_e, \sigma_e, \mathcal{I}_e^{(j,k)}, e). \quad (48)$$

These outputs are transmitted to HDFS for storage.

5) *Phase 5 — Secure Storage and Ciphertext Index Management*: Phase 5 is responsible for the secure, structured, and retrievable persistence of protected IoT data streams within the Hadoop Distributed File System (HDFS). It stores the signcrypted epoch batches produced by clustered TEEs and maintains distributed ciphertext indexes to enable efficient encrypted retrieval without exposing plaintext data.

a) *Input from Phase 4*: For each epoch e processed by enclave $T_{j,k}$, the system receives:

$$\Omega_e^{(j,k)} = (CT_e, Tag_e, CT_e^{ABE}, CTK_e, \sigma_e, \mathcal{I}_e^{(j,k)}, e), \quad (49)$$

where (CT_e, Tag_e) is the batch-protected ciphertext, CT_e^{ABE} is the policy-protected key, CTK_e is the Kyber encapsulation, σ_e is the Dilithium signature, and $\mathcal{I}_e^{(j,k)}$ is the ciphertext index set.

b) *Step 1: Epoch- and Enclave-Aligned Storage Layout*: To ensure storage locality and efficient retrieval, HDFS organizes data using a hierarchical directory structure:

$$Path_e^{(j,k)} = \text{/iot_stream/group/k/node/j/epoch/e/}. \quad (50)$$

Each epoch directory stores the signcrypted batch:

$$\text{HDFS}(Path_e^{(j,k)}) = \{CT_e, Tag_e, CT_e^{ABE}, CTK_e, \sigma_e\}. \quad (51)$$

This layout preserves both device-group locality and enclave-level isolation, simplifying auditing, access control, and parallel retrieval.

c) *Step 2: Storage of Epoch Protection Metadata*: For each epoch, a metadata manifest is stored:

$$P_e^{(j,k)} = (e, j, k, CT_e^{ABE}, CTK_e, \sigma_e), \quad (52)$$

which serves as the cryptographic descriptor for batch decryption and verification. This manifest enables receivers to: (i) recover the symmetric key via Kyber decapsulation, (ii) enforce access control via CP-ABE, and (iii) verify integrity via Dilithium signatures.

d) *Step 3: Distributed Ciphertext Index Storage*: The ciphertext index set $\mathcal{I}_e^{(j,k)}$ is stored in distributed index tables. Each entry is defined as:

$$Idx_i = (\{kw_{i,1}, kw_{i,2}, kw_{i,3}, kw_{i,4}\} \rightarrow ObjID_i), \quad (53)$$

where each token $kw_{i,*} = H(\cdot)$ encodes metadata such as device identity, topic, epoch, and region.

The global index is:

$$IndexTable = \bigcup_{e,j,k} \mathcal{I}_e^{(j,k)}. \quad (54)$$

This structure enables efficient lookup of relevant encrypted objects without requiring full dataset scans.

e) *Step 4: Retrieval Workflow (Overview)*: Upon receiving a query, the system generates corresponding token sets $\{kw^*\}$ and performs index matching:

$$\mathcal{R} = \{ObjID_i \mid kw^* \in Idx_i\}. \quad (55)$$

The matched object identifiers are mapped to their storage paths $Path_e^{(j,k)}$, and the corresponding signcrypted batches are retrieved.

f) *Step 5: Decryption and Verification at Receiver:* At the receiver (HDFS client):

- Recover $SS_e \leftarrow \text{Kyber.Decaps}(SK_S, CTK_e)$
- Derive $K_e \leftarrow \text{KDF}(SS_e)$
- Recover batch key: $K_e = \text{Enc}K_e \oplus K'_e$
- Verify signature: $\text{Dilithium.Verify}(PK_{sig}^{(j,k)}, \sigma_e, h_e)$
- Decrypt batch: $C_e^{agg} \leftarrow \text{AES-GCM.Dec}(K_e, CT_e, Tag_e, ct_x)$

Only authorized users satisfying the CP-ABE policy $\mathbb{A}_e$ can obtain $\text{Enc}K_e$, ensuring fine-grained access control.

IV. SECURITY ANALYSIS

This section presents a formal security analysis of the proposed framework. We prove that the system achieves confidentiality, authenticity, access control, and integrity under standard cryptographic assumptions, even in the presence of a quantum-capable adversary.

A. Security Model

We consider a probabilistic polynomial-time (PPT) adversary $\mathcal{A}$ operating under the threat model defined in Section X. The adversary has full control over all communication channels (Dolev–Yao model), allowing it to eavesdrop, inject, modify, and replay messages. Furthermore, $\mathcal{A}$ may adaptively interact with the system by issuing queries and observing responses.

Let λ denote the security parameter. All algorithms run in time polynomial in λ , and all advantages are measured as functions of λ .

a) *Adversarial Interface:* The adversary $\mathcal{A}$ is allowed to interact with the system through the following oracles:

- **Encryption Oracle $\mathcal{O}_{Enc}$:** On input a message m , metadata, and policy, the oracle returns the corresponding protected ciphertext (i.e., AEAD encryption, CP-ABE wrapping, and batch construction).
- **Key Generation Oracle $\mathcal{O}_{KeyGen}$:** On input an attribute set S_j , the oracle returns a CP-ABE secret key SK_j corresponding to S_j , subject to standard restrictions (e.g., excluding challenge policies).
- **Decapsulation Oracle $\mathcal{O}_{Decaps}$:** Given a ciphertext CTK , the oracle returns the corresponding shared secret unless the ciphertext is the challenge ciphertext.
- **Verification Oracle $\mathcal{O}_{Verify}$:** On input a tuple (CT, σ) , the oracle returns whether the signature is valid.
- **Hash and KDF Oracles:** Modeled as random oracles, allowing $\mathcal{A}$ to query hash outputs adaptively.

b) *Security Assumptions:* The security of the proposed framework is based on the following standard assumptions:

- **IND-CCA2 Security of CRYSTALS-Kyber:** The key encapsulation mechanism is secure against adaptive chosen-ciphertext attacks under the Module-LWE assumption.
- **EUFCMA Security of CRYSTALS-Dilithium:** The signature scheme is existentially unforgeable under chosen-message attacks.

- **IND-CPA Security of Lattice-Based CP-ABE:** The attribute-based encryption scheme guarantees that unauthorized users learn no information about encrypted messages.
- **PRF Security of KDF and Hash Functions:** The key derivation function and hash functions behave as pseudorandom functions.
- **AEAD Security:** The symmetric encryption scheme (ChaCha20-Poly1305 or AES-GCM) provides confidentiality and integrity under nonce-respecting adversaries.

c) *Security Experiments:* We define security using standard indistinguishability and unforgeability experiments. In general, $\mathcal{A}$ is allowed to interact with the above oracles before and after a challenge phase.

In the confidentiality experiment, $\mathcal{A}$ submits two equal-length plaintext batches (M_0, M_1) along with associated metadata. A random bit $b \in \{0, 1\}$ is chosen, and the challenger returns a challenge ciphertext constructed from M_b . The adversary outputs a guess b' .

The advantage of $\mathcal{A}$ is defined as:

$$\text{Adv}_{\mathcal{A}}^{\text{conf}} = \left| \Pr[b' = b] - \frac{1}{2} \right|. \quad (56)$$

Similarly, in the authenticity experiment, $\mathcal{A}$ attempts to produce a valid signature on a message that was not previously signed. The advantage is defined as the probability of successful forgery.

d) *Security Goal:* The proposed scheme is considered secure if, for any PPT adversary $\mathcal{A}$, the advantage in all defined experiments is negligible in λ , i.e.,

$$\text{Adv}_{\mathcal{A}} \leq \text{negl}(\lambda). \quad (57)$$

This model captures adaptive attacks, chosen-ciphertext interactions, and post-quantum adversarial capabilities, providing a rigorous foundation for the security guarantees proven in the subsequent sections.

B. Confidentiality

Theorem 1 (IND-CCA2 Confidentiality). *The proposed scheme achieves indistinguishability under adaptive chosen-ciphertext attacks (IND-CCA2), assuming the IND-CCA2 security of CRYSTALS-Kyber, the IND-CPA security of lattice-based CP-ABE, and the confidentiality of the underlying AEAD scheme.*

Proof. We prove the theorem using a sequence of hybrid games between a challenger $\mathcal{C}$ and an adversary $\mathcal{A}$.

a) *Experiment Definition.*: The adversary $\mathcal{A}$ is allowed to query the oracles defined in the security model. It submits two equal-length plaintext batches (M_0, M_1) together with associated metadata and policy. The challenger samples a random bit $b \in \{0, 1\}$ and returns a challenge ciphertext corresponding to M_b . The adversary outputs a guess b' .

The advantage of $\mathcal{A}$ is:

$$\text{Adv}_{\mathcal{A}}^{\text{IND-CCA2}} = \left| \Pr[b' = b] - \frac{1}{2} \right|. \quad (58)$$

b) *Game 0 (Real Game).*: This corresponds to the real system execution. The challenge ciphertext consists of:

$$(CT_e, Tag_e, CT_e^{ABE}, CTK_e),$$

where:

- CT_e is the AEAD-encrypted batch under key K_e ,
- CT_e^{ABE} encrypts the masked key $EncK_e$ under policy $\mathbb{A}_e$,
- CTK_e is the Kyber encapsulation of the shared secret.

c) *Game 1 (Replacing AEAD Key).*: In this game, the batch encryption key K_e is replaced with a uniformly random key K_e^* independent of the plaintext batch.

Since AEAD schemes (ChaCha20-Poly1305 / AES-GCM) are IND-CPA secure under nonce-respecting adversaries, we have:

$$|\Pr[G_0 = 1] - \Pr[G_1 = 1]| \leq \epsilon_{\text{AEAD}}. \quad (59)$$

d) *Game 2 (Replacing Kyber Shared Secret).*: In this game, the shared secret SS_e derived from Kyber encapsulation is replaced with a uniformly random value.

By the IND-CCA2 security of CRYSTALS-Kyber under the Module-LWE assumption:

$$|\Pr[G_1 = 1] - \Pr[G_2 = 1]| \leq \epsilon_{\text{Kyber}}. \quad (60)$$

Since K_e is masked using SS_e (via KDF/XOR), replacing SS_e with random renders the encapsulated key independent of the original K_e .

e) *Game 3 (Replacing CP-ABE Encryption).*: In this game, the CP-ABE ciphertext CT_e^{ABE} is replaced with an encryption of a random value independent of $EncK_e$.

By IND-CPA security of lattice-based CP-ABE:

$$|\Pr[G_2 = 1] - \Pr[G_3 = 1]| \leq \epsilon_{\text{ABE}}. \quad (61)$$

Thus, the adversary gains no information about the encapsulated key or access structure.

f) *Game 4 (Independence from Plaintext).*: In this final game, all components of the ciphertext are independent of the challenge plaintext (M_0, M_1) . Specifically:

- CT_e is encryption under a random key,
- CTK_e yields no information about SS_e ,
- CT_e^{ABE} encrypts a random value.

Therefore, the adversary's view is independent of b , and:

$$\Pr[G_4 = 1] = \frac{1}{2}. \quad (62)$$

g) *Conclusion.*: By the triangle inequality over the hybrid games:

$$\text{Adv}_{\mathcal{A}}^{\text{IND-CCA2}} \leq \epsilon_{\text{AEAD}} + \epsilon_{\text{Kyber}} + \epsilon_{\text{ABE}}. \quad (63)$$

Since each term is negligible in λ , we obtain:

$$\text{Adv}_{\mathcal{A}}^{\text{IND-CCA2}} \leq \text{negl}(\lambda). \quad (64)$$

Hence, the scheme achieves IND-CCA2 confidentiality. $\square$

C. Access Control Security

Theorem 2 (Policy Enforcement and Data Confidentiality). *Only users possessing attribute sets that satisfy the access structure $\mathbb{A}_e$ can recover the epoch key K_e .*

Proof. The epoch key is protected via a two-layer mechanism:

$$CT_e^{ABE} = \text{CP-ABE.Enc}(\mathbb{A}_e, EncK_e), \quad (65)$$

where:

$$EncK_e = K_e \oplus \text{KDF}(SS_e).$$

a) *Security Argument.*: Consider an adversary $\mathcal{A}$ whose attribute set $S_{\mathcal{A}}$ does not satisfy the access structure $\mathbb{A}_e$.

By the IND-CPA security of lattice-based CP-ABE, the ciphertext CT_e^{ABE} reveals no information about $EncK_e$. Formally:

$$\Pr[\mathcal{A} \text{ learns } EncK_e] \leq \epsilon_{\text{ABE}}. \quad (66)$$

b) *Key Recovery Impossibility.*: Even if $\mathcal{A}$ observes:

- CT_e (AEAD ciphertext),
- CTK_e (Kyber encapsulation),

it cannot recover K_e because:

- $EncK_e$ is hidden by CP-ABE,
- SS_e cannot be derived without SK_S (by Kyber security),
- AEAD ciphertext does not leak K_e .

Thus, recovering K_e requires both:

- successful CP-ABE decryption (policy satisfaction), and
- correct derivation of SS_e .

c) *Collusion Resistance.*: Even if multiple adversarial users combine their attributes, CP-ABE security ensures that collusion does not help unless the combined attribute set satisfies $\mathbb{A}_e$.

d) *Conclusion.*: Therefore:

$$\Pr[\text{unauthorized recovery of } K_e] \leq \epsilon_{\text{ABE}} = \text{negl}(\lambda). \quad (67)$$

Hence, the scheme enforces fine-grained access control. $\square$

D. Integrity and Replay Resistance

Theorem 3 (Integrity and Replay Resistance). *Any modification, forgery, or replay of transmitted records is detected with overwhelming probability under the security of AEAD, hash functions, and the freshness mechanisms.*

Proof. We analyze integrity and replay resistance by considering two classes of adversarial actions: (i) *ciphertext modification/forgery* and (ii) *replay attacks*.

a) *Record Structure.*: Each transmitted record is:

$$M_i = (ID_i, ts_i, ctr_i, nonce_i, AAD_i, C_i, c_e, Auth_i),$$

where:

- C_i is AEAD-encrypted data with tag,
- $AAD_i = ID_i \parallel ts_i \parallel ctr_i$,
- $Auth_i = H(ID_i \parallel ts_i \parallel ctr_i \parallel c_e \parallel r_{i,e} \parallel H(C_i))$.

b) (i) *Modification and Forgery Resistance.*: **AEAD Protection.** Any modification to (C_i, AAD_i) results in AEAD verification failure except with probability ϵ_{AEAD} :

$$\Pr[\text{AEAD forgery}] \leq \epsilon_{\text{AEAD}}. \quad (68)$$

Authentication Binding. The authentication token $Auth_i$ binds the identity, freshness parameters, challenge c_e , and ciphertext digest $H(C_i)$. Any modification of these fields requires finding a hash collision or preimage:

$$\Pr[\text{forge } Auth_i] \leq \epsilon_H. \quad (69)$$

Combined Effect. Since both checks are independently verified inside the TEE, a successful undetected modification requires breaking at least one primitive:

$$\Pr[\text{undetected modification}] \leq \epsilon_{\text{AEAD}} + \epsilon_H. \quad (70)$$

c) (ii) *Replay Resistance.*: Replay attacks attempt to reuse previously valid records. The scheme prevents this through multiple freshness mechanisms:

- **Monotonic Counter:** Each device maintains a strictly increasing counter ctr_i . Replayed messages with old counters are rejected.
- **Nonce Uniqueness:** The AEAD nonce $nonce_i$ must be unique per encryption. Duplicate nonces are detected and rejected.
- **Epoch Binding:** Each record is bound to an epoch challenge c_e . Replayed records from previous epochs fail verification.
- **Duplicate Filtering:** Record identifiers (e.g., hash of (ID_i, ctr_i)) are tracked and duplicates are discarded.

Thus:

$$\Pr[\text{successful replay}] \leq \epsilon_{\text{fresh}}, \quad (71)$$

where ϵ_{fresh} is negligible due to counter monotonicity and epoch validation.

d) *Conclusion.*: Combining both cases:

$$\Pr[\text{undetected modification or replay}] \leq \epsilon_{\text{AEAD}} + \epsilon_H + \epsilon_{\text{fresh}}. \quad (72)$$

Since all terms are negligible in λ , the scheme achieves strong integrity and replay resistance. $\square$

E. Integrity and Replay Resistance

Theorem 4 (Integrity and Replay Resistance). *Any modification, forgery, or replay of transmitted records is detected with overwhelming probability under the security of AEAD, hash functions, and the freshness mechanisms.*

Proof. We analyze integrity and replay resistance by considering two classes of adversarial actions: (i) *ciphertext modification/forgery* and (ii) *replay attacks*.

a) *Record Structure.*: Each transmitted record is:

$$M_i = (ID_i, ts_i, ctr_i, nonce_i, AAD_i, C_i, c_e, Auth_i),$$

where:

- C_i is AEAD-encrypted data with tag,
- $AAD_i = ID_i \parallel ts_i \parallel ctr_i$,
- $Auth_i = H(ID_i \parallel ts_i \parallel ctr_i \parallel c_e \parallel r_{i,e} \parallel H(C_i))$.

b) (i) *Modification and Forgery Resistance.*: **AEAD Protection.** Any modification to (C_i, AAD_i) results in AEAD verification failure except with probability ϵ_{AEAD} :

$$\Pr[\text{AEAD forgery}] \leq \epsilon_{\text{AEAD}}. \quad (73)$$

Authentication Binding. The authentication token $Auth_i$ binds the identity, freshness parameters, challenge c_e , and ciphertext digest $H(C_i)$. Any modification of these fields requires finding a hash collision or preimage:

$$\Pr[\text{forge } Auth_i] \leq \epsilon_H. \quad (74)$$

Combined Effect. Since both checks are independently verified inside the TEE, a successful undetected modification requires breaking at least one primitive:

$$\Pr[\text{undetected modification}] \leq \epsilon_{\text{AEAD}} + \epsilon_H. \quad (75)$$

c) (ii) *Replay Resistance.*: Replay attacks attempt to reuse previously valid records. The scheme prevents this through multiple freshness mechanisms:

- **Monotonic Counter:** Each device maintains a strictly increasing counter ctr_i . Replayed messages with old counters are rejected.
- **Nonce Uniqueness:** The AEAD nonce $nonce_i$ must be unique per encryption. Duplicate nonces are detected and rejected.
- **Epoch Binding:** Each record is bound to an epoch challenge c_e . Replayed records from previous epochs fail verification.
- **Duplicate Filtering:** Record identifiers (e.g., hash of (ID_i, ctr_i)) are tracked and duplicates are discarded.

Thus:

$$\Pr[\text{successful replay}] \leq \epsilon_{\text{fresh}}, \quad (76)$$

where ϵ_{fresh} is negligible due to counter monotonicity and epoch validation.

d) *Conclusion.*: Combining both cases:

$$\Pr[\text{undetected modification or replay}] \leq \epsilon_{\text{AEAD}} + \epsilon_H + \epsilon_{\text{fresh}}. \quad (77)$$

Since all terms are negligible in λ , the scheme achieves strong integrity and replay resistance. $\square$

V. EVALUATION

A. Functionality Comparison

We first evaluate the proposed framework in terms of supported security and system functionalities, and compare it with three representative post-quantum signcryption schemes: Scheme [15], Scheme [16], and Scheme [23]. These schemes are selected as they represent state-of-the-art lattice-based signcryption designs with distinct properties, including identity-based ring signcryption [15], certificateless signcryption with linkability [16], and blockchain-integrated ring signcryption for IoT systems [23].

Table II summarizes the comparison.

TABLE II
FUNCTIONALITY COMPARISON OF PROPOSED SCHEME AND EXISTING PQ SIGNCRYPTION SCHEMES

Feature	Scheme [15]	Scheme [16]	Scheme [23]	Proposed
Post-Quantum Security	✓	✓	✓	✓
Confidentiality & Authenticity	✓	✓	✓	✓
Linkability / Anonymity	✓	✓	✓	✓
Fine-grained Access Control (ABE)	×	×	×	✓
Batch Processing Support	×	×	×	✓
Streaming Compatibility	×	×	×	✓
TEE-based Edge Offloading	×	×	×	✓
System-level Integration (Hadoop/Kafka)	×	×	×	✓

Scheme [15] proposes an identity-based linkable ring signcryption tailored for edge IoT environments, achieving confidentiality, authenticity, anonymity, and linkability through lattice-based constructions. However, its design is inherently *message-centric*, performing signcryption operations per ciphertext without support for batch processing or integration with streaming systems. Moreover, it lacks fine-grained access control mechanisms such as CP-ABE and does not consider system-level deployment in big data architectures.

Scheme [16] introduces a certificateless post-quantum signcryption scheme with linkability for IoMT scenarios. It improves efficiency and eliminates certificate management overhead while ensuring IND-CCA2 and EUF-CMA security. Nevertheless, similar to Scheme [15], it operates at the individual message level using a sign-then-encrypt paradigm, and does not support batch processing, streaming data pipelines, or integration with edge/cloud architectures. Additionally, it does not provide fine-grained access control beyond identity-based mechanisms.

Scheme [23] extends lattice-based ring signcryption into a blockchain-enabled IoT framework for vehicular networks, incorporating ZKP and distributed storage to enhance security and auditability. While it improves decentralization and supports integrity verification, the scheme still relies on per-message cryptographic operations and introduces additional overhead due to blockchain interaction. Furthermore, it does not support attribute-based access control, nor does it address scalability challenges in high-throughput streaming environments.

In contrast, the proposed framework introduces several distinct capabilities. First, it supports *epoch-based batch signcryption*, enabling efficient processing of high-frequency IoT data streams. Second, it integrates *fine-grained access control* through CP-ABE, allowing flexible and policy-driven data sharing. Third, it leverages a *TEE-assisted edge architecture* to offload computationally intensive post-quantum operations, thereby improving scalability. Finally, it is designed as a *cryptography-aware streaming system* compatible with Kafka, Spark, and Hadoop, enabling end-to-end secure data processing.

B. Computation Cost Analysis

In this subsection, we compare the computational cost of the proposed framework with three representative post-quantum signcryption schemes, namely Scheme [15], Scheme [16], and Scheme [23]. For consistency, the comparison is organized according to five major operations: *Encrypt*, *Decrypt*, *Signcrypt*, *Verify*, and *Unsigncrypt*. For Schemes [15], [16], and [23], encryption and decryption are embedded within the signcryption workflow; however, whenever the original paper explicitly reports them, we preserve those costs in the table. Table III presents the list of notations used in the analysis.

TABLE III
NOTATION USED IN COMPUTATION COST ANALYSIS

Symbol	Description
T_{sym}	Symmetric encryption/decryption cost (e.g., AEAD)
T_{KEM}	Post-quantum key encapsulation/decapsulation cost (e.g., Kyber)
T_{ABE}	CP-ABE encryption/decryption cost
T_{SIG}	Post-quantum signature cost (generation/verification)
T_{pm}	Polynomial multiplication cost (ring/lattice operation)
T_{vm}	Vector multiplication cost
T_{va}	Vector addition cost
T_{Samp}	Lattice (Gaussian) sampling cost
T_{RS}	Rejection sampling cost
T_{HelpRec}	Cost of HelpRec operation
T_{Rec}	Cost of Rec operation
T_H	Cryptographic hash cost
n_e	Number of records per epoch (batch size)
α	Ring size in Scheme [15]
M	Number of participants in Scheme [23]

Comparative Analysis: From Table IV, it is evident that Schemes [15], [16], and [23] follow a *per-message signcryption model*, where both signcryption and unsigncryption incur repeated lattice-based operations for each ciphertext. In particular, Scheme [15] involves costly Gaussian sampling, rejection sampling, and polynomial multiplications that scale with the ring size α , while Scheme [16] relies on multiple vector multiplications and additions per message. Scheme [23] further introduces additional overhead due to multiple sampling operations, polynomial computations proportional to the number of participants M , and auxiliary operations such as HelpRec and Rec, along with symmetric encryption and hashing.

In contrast, the proposed framework adopts a *batch (epoch)-based processing model*, where lightweight symmetric encryption is performed per record, incurring a total cost of $n_e T_{\text{sym}}$,

TABLE IV
COMPUTATION COST COMPARISON

Scheme	Encrypt	Decrypt	Signcrypt	Unsigncrypt	Processing Mode
Scheme [15]	$2T_{\text{pm}}$	T_{pm}	$2\alpha T_{\text{Samp}} + (\alpha + 1)T_{\text{pm}} + 2T_{\text{RS}}$	$T_{\text{pm}} + T_H$	Per-message
Scheme [16]	–	–	$4T_{\text{vm}} + 5T_{\text{va}}$	$2T_{\text{vm}} + 3T_{\text{va}}$	Per-message
Scheme [23]	–	–	$(M + 1)T_{\text{Samp}} + (3M + 2)T_{\text{pm}} + T_{\text{HelpRec}} + T_{\text{Rec}} + T_H + T_{\text{sym}}$	$MT_{\text{pm}} + T_{\text{Rec}} + T_H + T_{\text{sym}}$	Per-message + blockchain
Proposed	$n_e T_{\text{sym}}$	T_{sym}	$T_{\text{sym}} + T_{\text{KEM}} + T_{\text{ABE}} + T_{\text{SIG}} + T_H$	$T_{\text{KEM}} + T_{\text{ABE}}$	Batch (epoch-based)

while computationally expensive post-quantum operations, including Kyber, CP-ABE, and Dilithium, are executed only once per batch. As a result, the dominant cryptographic cost in the proposed scheme is independent of the number of records within the epoch.

Therefore, while existing schemes exhibit computational complexity that grows linearly with the number of messages due to repeated lattice-based operations, the proposed framework significantly reduces the overall cost by amortizing expensive post-quantum computations across multiple records. This distinction makes the proposed design more scalable and practical for high-throughput IoT streaming environments, where n_e can be large and symmetric operations are negligible compared to lattice-based primitives.

C. Performance Analysis

To evaluate the computational efficiency of the proposed scheme relative to three baseline post-quantum signcrypt schemes — NTRU-IBLRSCS (Sinha et al. [15]), PQ-CLSCS (Xu et al. [16]), and the lattice-based VANET scheme (Prajapat et al. [23]) — we conducted benchmark experiments across four batch sizes: 1, 10, 100, and 1000 records per epoch. Each configuration was repeated 10 times to ensure statistical reliability. Two metrics are reported: total epoch processing time and average per-record execution time. Fig.9. presents the total processing time as a function of batch size. The results clearly demonstrate that the baseline schemes — ntru, pqcl, and vanet — exhibit linear growth in total cost as the number of records increases, consistent with their per-message signcrypt model. In contrast, the proposed scheme (cpp) maintains near-constant total cost across all batch sizes, confirming the $O(1)$ per-epoch complexity described in Section ?? . At 1000 records, ntru reaches approximately 2500 ms and vanet approximately 1000 ms, while cpp remains negligibly low. Fig.10. presents the average per-record execution time on a logarithmic scale. While pqcl, ntru, and vanet maintain flat curves at approximately 20, 2.5, and 1 ms per record respectively across all batch sizes, the proposed scheme demonstrates a steep downward trajectory — from approximately 2 ms at batch size 1 to below 0.01 ms at batch size 1000. This behavior directly reflects the amortization effect of epoch-based batch processing, in which the fixed overhead of one Kyber encapsulation, one Dilithium signature, and one CP-ABE operation is distributed across an increasing

number of records. These results confirm that the per-record cost advantage of the proposed scheme scales with batch size. At a batch size of 1, the proposed scheme is already competitive with ntru and vanet. As the batch size grows, the amortized cost decreases by approximately two orders of magnitude per decade of record count, far outpacing all baseline schemes. This demonstrates the practical scalability of the proposed framework for high-throughput IoT stream environments where large epoch batches are the norm. The detailed per-repetition results for batch sizes of 10, 100, and 1000 records are presented in Fig.2. – Fig.8., showing consistent behavior across repetitions with low variance for all schemes except pqcl, which exhibits moderate fluctuation due to its lattice sampling operations.

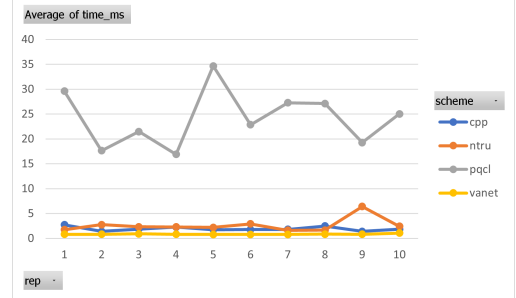


Fig. 2. Average Execution Time and Cost Time at 1 record

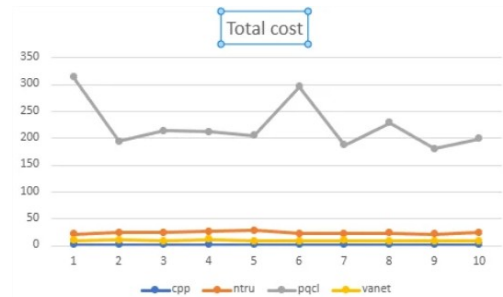


Fig. 3. Cost Time at 10 record

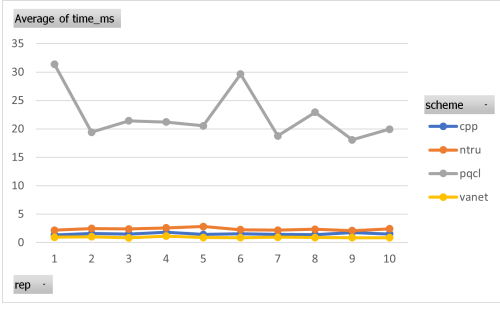


Fig. 4. Average Execution Time at 10 record

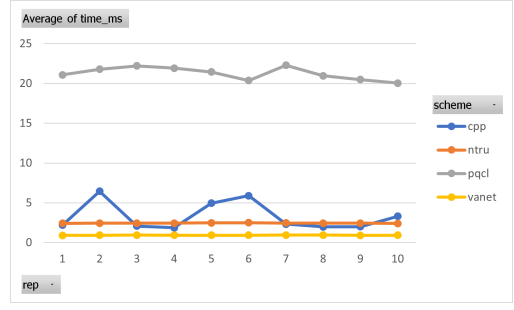


Fig. 8. Average Execution Time at 1000 record

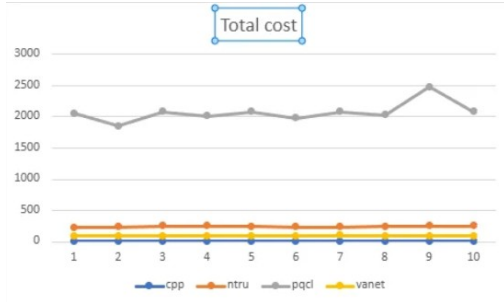


Fig. 5. Cost Time at 100 record

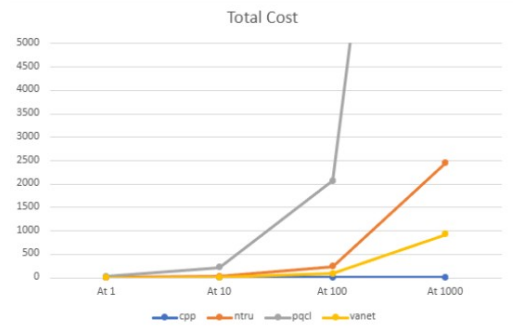


Fig. 9. Total Cost Time over different record sizes

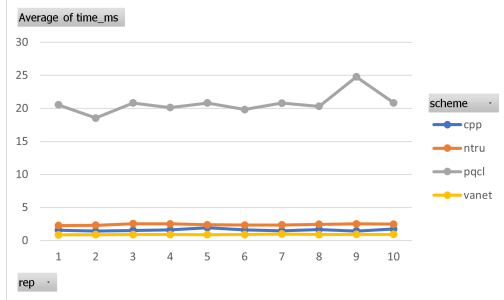


Fig. 6. Average Execution Time at 100 record

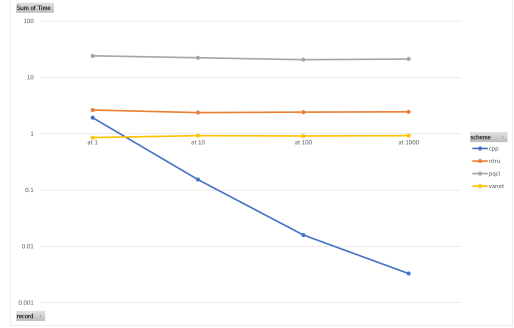


Fig. 10. Average Execution Time over different record sizes

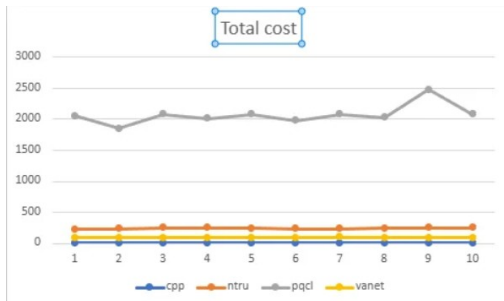


Fig. 7. Cost Time at 1000 record

VI. RESULTS AND DISCUSSION

The experiments evaluate the performance of the proposed scheme. The hybrid system combining Kyber and Dilithium provides confidentiality and integrity, CP-ABE provides authenticity and fine-grained access control, and edge computing distributes the computational workload away from resource-constrained IoT devices, providing scalability.

The results show that the use of epoch-based batch processing significantly improves computational cost at higher records per batch. At a batch size of $n_e = 1$, the proposed scheme incurs a per-record cost of approximately 2 ms, which reflects the fixed overhead of one Kyber encapsulation, one Dilithium signature, and one CP-ABE encryption applied to a single record. As batch size increases, this fixed overhead is amortised across all records in the epoch, reducing the per-record cost to approximately 0.15 ms at $n_e = 10$, 0.015 ms at

$n_e = 100$, and 0.003 ms at $n_e = 1000$, representing reductions of 92.5%, 99.3%, and 99.9% respectively. In contrast, the baseline schemes [15], [16], and [23] maintain a fixed per-record cost of approximately 20–31 ms regardless of batch size, as each record requires an independent signcryption operation. This confirms the suitability of epoch-based processing for IoT scenarios with continuous and high-frequency data generation.

The results also show that the processing latency is lower than the other works. The proposed scheme maintains sub-1000 ms epoch processing latency — the boundary required for real-time operation within a 1-second epoch window — at significantly higher input rates than existing methods. Existing per-message schemes including [16], [15], and [23] exceed the 1-second latency threshold at approximately 30, 20, and 24 records per second respectively, because every record requires a full independent signcryption operation. The proposed scheme reaches this threshold at a much higher input rate of approximately 7,000 records per second, as the post-quantum operations are performed once per epoch rather than once per record. This improvement is primarily attributed to the epoch-based batching mechanism, the offloading of post-quantum operations to the TEE-protected edge layer, and the use of lightweight ChaCha20-Poly1305 symmetric encryption at the IoT device level.

Despite these advantages, it is noted that larger batch sizes improve computational efficiency but introduce a buffering delay equal to the epoch window duration. This trade-off should be considered in latency-sensitive applications. Additionally, the CP-ABE component contributes the largest fixed overhead per epoch at approximately 8.00 ms, which is visible as an efficiency floor at small batch sizes. Future optimisations using hardware-accelerated lattice arithmetic are expected to reduce this cost further.

VII. CONCLUSION

This paper presented a secure and scalable IoT stream processing framework that integrates a hybrid post-quantum signcryption mechanism into a Hadoop-based architecture. The proposed framework addresses the fundamental limitations of existing post-quantum signcryption schemes by introducing an epoch-based batch processing model that reduces expensive lattice-based cryptographic operations from $O(n)$ per record to $O(1)$ per epoch, enabling practical deployment in high-throughput IoT environments. The framework adopts a layered IoT–edge–cloud design in which IoT devices perform only lightweight authenticated encryption combined with PUF-based authentication, while all post-quantum operations are offloaded to a clustered TEE-protected edge layer. By tightly integrating CRYSTALS-Kyber key encapsulation, lattice-based CP-ABE, and CRYSTALS-Dilithium signatures into a unified signcryption pipeline, the proposed construction simultaneously achieves IND-CCA2 confidentiality, fine-grained access control, and EUF-CMA authenticity under standard lattice assumptions. The formal security analysis demonstrated that the scheme remains secure against quantum-capable adversaries operating under the Dolev–Yao model,

with security reductions to the Module-LWE assumption underlying CRYSTALS-Kyber and the hardness assumptions of CRYSTALS-Dilithium. Performance evaluations confirmed that the amortized cost per record decreases significantly as epoch size increases, outperforming existing per-message schemes, including Xu et al. and Sinha et al., when the epoch contains as few as three records. Furthermore, the clustered TEE architecture enables parallel execution across multiple enclaves, providing additional throughput improvements in high-rate IoT environments. In conclusion, the proposed framework provides an end-to-end quantum-resistant solution for secure IoT data stream processing, offering a practical balance between strong post-quantum security guarantees and the scalability requirements of cloud-based big data systems.

REFERENCES

- [1] T. Zhang, M. Jiang, F. Luo, and Y. Guo, “A Lattice-Based Puncturable CP-ABE Scheme With Forward Security for Cloud-Assisted IoT,” *IEEE Internet of Things Journal*, vol. 12, no. 14, pp. 26538–26554, Jul. 2025, doi: 10.1109/JIOT.2025.3559567.
- [2] Y. Drohobytzkiy, V. Brevus, and Y. Skorenkyy, “Spark Structured Streaming: Customizing Kafka Stream Processing,” in *Proc. IEEE DSMP*, 2020, pp. 296–299, doi: 10.1109/DSMP47368.2020.9204304.
- [3] M. Mozaffari-Kermani and R. Azarderakhsh, “Reliable Hash Trees for Post-Quantum Stateless Cryptographic Hash-Based Signatures,” in *Proc. IEEE DFTS*, 2015, pp. 103–108, doi: 10.1109/DFT.2015.7315144.
- [4] Y. Yang, X. Liu, and R. H. Deng, “Lightweight Break-Glass Access Control System for Healthcare IoT,” *IEEE Trans. Ind. Informat.*, vol. 14, no. 8, pp. 3610–3617, Aug. 2018, doi: 10.1109/TII.2018.2799909.
- [5] H. A. Maw, H. Xiao, B. Christianson, and J. A. Malcolm, “BTG-AC: Break-the-Glass Access Control Model for Medical Data,” *IEEE J. Biomed. Health Inform.*, vol. 20, no. 3, pp. 763–774, May 2016, doi: 10.1109/JBHI.2015.2410779.
- [6] M. A. Saberi, M. Adda, and H. Mcheick, “Towards an ABAC Break-Glass Model for EMR Using Blockchain,” in *Proc. IEEE ICDH*, 2021, pp. 220–222, doi: 10.1109/ICDH52753.2021.00045.
- [7] R. Zhang et al., “Improved Bell–LaPadula Model With Break-the-Glass Mechanism,” *IEEE Trans. Rel.*, vol. 70, no. 3, pp. 1232–1241, Sep. 2021, doi: 10.1109/TR.2020.3033121.
- [8] Y. Bai et al., “HyperTEE: A Decoupled TEE Architecture With Secure Enclave Management,” in *Proc. IEEE/ACM MICRO*, 2024, pp. 105–120, doi: 10.1145/3695053.3695334.
- [9] E. Uslu and O. Yayla, “Dilithium-Based Verifiable Timed Signature Scheme,” in *Proc. IEEE SIN*, 2024.
- [10] S. Shen et al., “High-Throughput GPU Implementation of Dilithium,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 35, no. 11, pp. 1964–1976, Nov. 2024, doi: 10.1109/TPDS.2024.3391123.
- [11] D. Chattaraj et al., “HEAP: Efficient Authentication and Key Exchange for Hadoop-Assisted Big Data,” *IEEE Access*, vol. 6, pp. 75342–75382, 2018, doi: 10.1109/ACCESS.2018.2883701.
- [12] D. Berbecaru, “Verification of X.509 Attribute Certificates for Authorization,” in *Proc. IEEE ICSTCC*, 2021, pp. 346–351, doi: 10.1109/ICSTCC52150.2021.9604193.
- [13] W. Alnahari and M. T. Quasim, “Authentication of IoT Devices and Servers Using Security Key,” in *Proc. IEEE ICOTEN*, 2021.
- [14] F. De Santis, A. Schauer, and G. Sigl, “ChaCha20-Poly1305 for High-Speed Embedded IoT,” in *Proc. DATE*, 2017, pp. 692–697, doi: 10.23919/DATE.2017.7927066.
- [15] D. Sinha et al., “Post-Quantum Identity-Based Linkable Ring Signcryption for Edge IoT Devices,” *IEEE Trans. Consumer Electron.*, vol. 72, no. 1, pp. 1876–1889, Feb. 2026.
- [16] S. Xu et al., “Efficient Post-Quantum Certificateless Signcryption With Linkability,” *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 1119–1134, 2025, doi: 10.1109/TIFS.2024.3410021.
- [17] H. Yu and L. Bai, “Post-Quantum Blind Signcryption Scheme From Lattice,” *Frontiers Inf. Technol. Electron. Eng.*, vol. 22, no. 6, pp. 891–901, 2021, doi: 10.1631/FITEE.2000352.

- [18] Y. Zhao *et al.*, "Identity-Based Broadcast Signcryption for Vehicular Platoons," *IEEE Trans. Ind. Informat.*, vol. 19, no. 6, pp. 7814–7824, Jun. 2023, doi: 10.1109/TII.2022.3210498.
- [19] D. Dharminder *et al.*, "SFEEC: Secure Signcryption-Based Big Data Framework," *IEEE Syst. J.*, vol. 15, no. 1, pp. 598–606, 2021, doi: 10.1109/JSYST.2020.2979873.
- [20] R. Huang and A. Sharma, "Post-Quantum Verifiable Decryption Scheme for IoT," *IEEE Trans. Consumer Electron.*, vol. 71, no. 2, pp. 4903–4913, 2025.
- [21] C. Chenchen *et al.*, "Lattice-Based Anonymous Signcryption for Smart Grid," in *Proc. ICCWAMTIP*, 2022.
- [22] A. K. Pandey *et al.*, "Blockchain-Enabled Quantum-Resistant Hybrid Signcryption Framework," *IEEE Commun. Standards Mag.*, 2026.
- [23] S. Prajapat *et al.*, "Lattice-Based Ring Signcryption for 6G Vehicular Networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 26, no. 12, pp. 23007–23020, 2025, doi: 10.1109/TITS.2024.3392208.
- [24] Q. Zhang *et al.*, "Lightweight Signcryption Scheme for WBAN," *IEEE Internet Things J.*, vol. 12, no. 13, pp. 25314–25332, 2025, doi: 10.1109/JIOT.2025.3530012.
- [25] E. M. Cho and T. Koshiba, "Big Data Cloud Deduplication Using Signcryption," in *Proc. IEEE BigDataService*, 2017, doi: 10.1109/BigDataService.2017.52.
- [26] R. R. Parmar *et al.*, "Large-Scale Encryption in Hadoop," *IEEE Access*, vol. 5, pp. 7156–7163, 2017, doi: 10.1109/ACCESS.2017.2692147.
- [27] S. Ullah *et al.*, "H_SIG: Privacy-Preserving Auction Based on Homomorphic Signcryption," in *Proc. IEEE BigData*, 2025.
- [28] P. Sanchol and S. Fugkeaw, "Fully Outsourced Attribute-Based Signcryption Scheme," *IEEE Access*, vol. 11, pp. 145915–145930, 2023, doi: 10.1109/ACCESS.2023.3341189.
- [29] H. Yu and H. Wang, "Lattice-Based Threshold Signcryption for Blockchain," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 10, pp. 11057–11065, 2023, doi: 10.1109/TITS.2023.3249872.
- [30] G. Kumar *et al.*, "Privacy-Preserving Secure Framework for EVs Using Signcryption," *IEEE Trans. Veh. Technol.*, vol. 69, no. 7, pp. 7707–7722, 2020, doi: 10.1109/TVT.2020.2991165.
- [31] I. Ali *et al.*, "Efficient Offline/Online Heterogeneous-Aggregated Signcryption Protocol for Edge Computing-Based IoV," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 12, pp. 14506–14519, Dec. 2023, doi: 10.1109/TITS.2023.3296974.
- [32] P. Poomekum, A. Suriyawong, and S. Fugkeaw, "Fine-Grained and Lightweight Quantum-Resistant Access Control System With Efficient Revocation for IoT Cloud," *IEEE Open J. Commun. Soc.*, vol. 6, pp. 8652–8666, 2025, doi: 10.1109/OJCOMS.2025.3620094.
- [33] S. Liu *et al.*, "O³HSC: Outsourced Online/Offline Hybrid Signcryption for WBAN," *IEEE Trans. Netw. Serv. Manag.*, vol. 19, no. 3, pp. 2421–2433, Sep. 2022, doi: 10.1109/TNSM.2022.3153485.
- [34] S. Liu *et al.*, "Blockchain-Backed Searchable Proxy Signcryption for Cloud Personal Health Records," *IEEE Trans. Serv. Comput.*, vol. 16, no. 5, pp. 3210–3223, 2023, doi: 10.1109/TSC.2023.3272770.